

Guía de estudio

Programación Avanzada — Primer examen parcial

Prof. Gustavo Márquez Flores · Semestre 2026-1

CUADERNO DE PROBLEMAS

Resuelve primero sin mirar; después compara con el cuaderno de soluciones (misma numeración).

Contenido

1. Fundamentos: lenguajes, gramáticas, paradigmas, programación estructurada y modular, métricas de módulos.
2. Divide y vencerás: esquema general, búsqueda binaria, QuickSort, multiplicación de enteros grandes.
3. Backtracking: una, todas y la mejor solución; ocho reinas; cuadros mágicos y construcción por patrones.
4. Lógica de Hoare básica: aserciones, fortaleza, corrección parcial/total, asignación, consecuencia, conjunción.
5. Composición secuencial e invariantes de bloque.
6. Condicionales (if con y sin **else**).
7. Ciclos: inducción, invariante de ciclo, salida y terminación.
8. Recursión doble: función 91 de McCarthy, inducción modificada y función de Ackermann.
9. Programación lógica: cláusulas de Horn, resolución, SLD, unificación, backtracking y corte en Prolog.
10. Dos exámenes simulados (formato de 5–6 preguntas, como el del año pasado).

Los problemas marcados con ★ son *literalmente* las preguntas del examen del año pasado.

Cómo es el examen del profesor

El primer parcial del año pasado tuvo **cinco preguntas** que combinan demostración formal, teoría breve y pseudocódigo:

#	Pregunta	Tipo
1	Demostrar que $\{a < b\}$ if $(a > b)$ then $m := a$ else $m := b$ $\{m = b\}$ es válida.	Demostración (Hoare)
2	¿Qué métricas de módulos existen para medir su eficiencia?	Teoría breve
3	Escribir en pseudocódigo el algoritmo de <i>Divide y vencerás</i> .	Pseudocódigo
4	Demostrar que el invariante de <code>int j=9; for(int i=0; i<10; i++) j-;</code> es $i + j = 9$.	Demostración (inducción)
5	Describir el algoritmo de <i>Backtracking</i> para buscar la <i>mejor</i> solución.	Pseudocódigo + teoría

Qué se deduce: el profesor pide (i) demostraciones con reglas de Hoare escritas completas, (ii) invariantes de ciclos probados por inducción, (iii) los esquemas de sus hojas “*Algoritmo BackTracking y Divide-Vencerás*” en su misma notación y (iv) definiciones cortas. Como este año el curso ya avanzó a la función 91, Ackermann y Prolog, es razonable esperar al menos una pregunta de esos temas.

Lección del examen corregido

En la pregunta 1 el alumno supuso $a > b$ y trató de “forzar” $m = b$; el profesor anotó “revisar planteamiento”. La clave era **reconocer que la rama then es inalcanzable**: $a < b \wedge a > b \equiv \perp$, y una terna con precondition $\perp$ es válida por vacuidad. Siempre escribe explícitamente P , B , Q , las dos ramas y la regla que usas.

Plantillas que conviene memorizar

Plantilla: demostrar una terna con condicional

1. Identificar P (precondición), B (guarda) y Q (postcondición).
2. **Rama verdadera:** con precondition $P \wedge B$, calcular $wp(C_1, Q)$ por sustitución y probar $P \wedge B \Rightarrow wp(C_1, Q)$ (regla de consecuencia). Si $P \wedge B \equiv \perp$, la rama es inalcanzable y la terna vale por vacuidad.
3. **Rama falsa:** igual con $P \wedge \neg B$ y C_2 (si no hay **else**: probar $P \wedge \neg B \Rightarrow Q$).
4. Concluir con la regla condicional. Sin ciclos ni recursión, el código termina: corrección total.

Plantilla: demostrar un invariante de ciclo

1. Traducir el `for` a `while` y nombrar valores tras n iteraciones: x_n .
2. **Caso base** ($n = 0$): el invariante I vale antes de la primera iteración.
3. **Hipótesis de inducción:** I vale tras n iteraciones.
4. **Paso inductivo:** si la guarda B es cierta y se ejecuta el cuerpo, I vale tras $n + 1$ iteraciones

(equivalente: $\{I \wedge B\}$ cuerpo $\{I\}$).

5. **Salida:** $I \wedge \neg B \Rightarrow Q$. **Terminación:** variante entera ≥ 0 que decrece.

Formulario de repaso

Lógica de Hoare

Terna	$\{P\} C \{Q\}$: si C inicia en un estado que satisface P y <i>termina</i> , el estado final satisface Q .
Corrección	total = parcial + terminación.
Fortaleza	R es más fuerte que S si $R \Rightarrow S$. $\perp$ (ningún estado) es la más fuerte; $\top \equiv \{ \}$ (todos) la más débil.
Asignación	$\{Q[E/V]\} V := E \{Q\}$ (sustituir V por E en Q ; añadir condiciones de dominio, p. ej. divisor $\neq 0$).
Consecuencia	$\frac{P_1 \Rightarrow P \quad \{P\} C \{Q\}}{\{P_1\} C \{Q\}} \quad \frac{\{P\} C \{Q\} \quad Q \Rightarrow Q_1}{\{P\} C \{Q_1\}}$
Conjunción / disyunción	$\frac{\{P_1\} C \{Q_1\} \quad \{P_2\} C \{Q_2\}}{\{P_1 \wedge P_2\} C \{Q_1 \wedge Q_2\}} \quad \frac{\{P_1\} C \{Q_1\} \quad \{P_2\} C \{Q_2\}}{\{P_1 \vee P_2\} C \{Q_1 \vee Q_2\}}$
Composición	$\frac{\{P\} C_1 \{R\} \quad \{R\} C_2 \{Q\}}{\{P\} C_1; C_2 \{Q\}} \quad (R: \text{aserción intermedia; se calcula hacia atrás}).$
if sin else	$\frac{\{P \wedge B\} C_1 \{Q\} \quad (P \wedge \neg B) \Rightarrow Q}{\{P\} \text{ if } B \text{ then } C_1 \{Q\}}$
if con else	$\frac{\{P \wedge B\} C_1 \{Q\} \quad \{P \wedge \neg B\} C_2 \{Q\}}{\{P\} \text{ if } B \text{ then } C_1 \text{ else } C_2 \{Q\}}$
while	$P \Rightarrow I; \{I \wedge B\} C \{I\}; I \wedge \neg B \Rightarrow Q$; variante $V \in \mathbb{N}$ que decrece estrictamente.
Invariante de bloque	$\{I\} C \{I\}$: se conserva la <i>relación</i> , no los valores.

Inducción, McCarthy y Ackermann

Inducción ordinaria: $P(k)$ y $\forall n \geq k (P(n) \Rightarrow P(n+1)) \Rightarrow \forall n \geq k P(n)$.

Inducción modificada (hacia abajo): $P(k)$ y, para todo $n < k$, $[P(m)$ para todo m con $n < m \leq k] \Rightarrow P(n) \Rightarrow \forall n \leq k P(n)$.

Función 91: $f(x) = x - 10$ si $x > 100$; $f(x) = f(f(x+11))$ si $x \leq 100$. Teorema: $f(x) = 91$ para $x \leq 100$.

Ackermann: $A(0, y) = y + 1$; $A(x+1, 0) = A(x, 1)$; $A(x+1, y+1) = A(x, A(x+1, y))$.

$A(1, z) = z + 2$, $A(2, z) = 2z + 3$, $A(3, z) = 2^{z+3} - 3$, $A(4, z) = \underbrace{2^{2^{\cdot^{\cdot^2}}}}_{z+3 \text{ doses}} - 3$ ($A(4, 0) = 13$, $A(4, 1) = 65533$).

Técnicas de diseño

Divide y vencerás: caso base simple; descomponer en k subproblemas *semejantes y menores*; resolver recursivamente; *combinar*.

Recurrencias típicas: $T(n) = T(n/2) + \Theta(1) \Rightarrow \Theta(\log n)$ (búsqueda binaria); $2T(n/2) + \Theta(1) \Rightarrow \Theta(n)$ (máximo); $2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$ (mergesort, QuickSort balanceado); $T(n-1) + \Theta(n) \Rightarrow \Theta(n^2)$ (QuickSort peor caso); $4T(n/2) + \Theta(n) \Rightarrow \Theta(n^2)$; $3T(n/2) + \Theta(n) \Rightarrow \Theta(n^{\log_2 3})$ (Karatsuba).

Backtracking: árbol de decisiones recorrido en profundidad; *elegir – validar (poda) – anotar – avanzar – deshacer*. *Una*: se detiene con el primer éxito. *Todas*: registra y sigue. *Mejor*: compara con la mejor guardada y sigue (con cota $\rightarrow$ *branch and bound*).

Ocho reinas: una reina por fila; columna j , diagonales $j-k$ y $j+k$. 92 soluciones (12 fundamentales). $n=4$: 2 soluciones.

Cuadro mágico: $M_n = \frac{n(n^2+1)}{2}$ ($M_3 = 15$, $M_4 = 34$, $M_5 = 65$). Impar: método siamés (arriba-derecha; si está ocupada, abajo). $n \equiv 0 \pmod{4}$: llenar $1..n^2$ y cambiar $x \mapsto n^2 + 1 - x$ fuera de las diagonales de cada bloque 4×4 .

Prolog y resolución

Regla $a :- b_1, \dots, b_m. \equiv b_1 \wedge \dots \wedge b_m \rightarrow a \equiv a \vee \neg b_1 \vee \dots \vee \neg b_m$ (cláusula de Horn definida).

Meta $?- a_1, \dots, a_n. \equiv \leftarrow A_1, \dots, A_n \equiv \neg A_1 \vee \dots \vee \neg A_n$.

Resolución: $\frac{C \vee L \quad D \vee \neg L'}{(C \vee D)\theta}$, $\theta = \text{UMG}(L, L')$. Refutación: $S \cup \{\neg F\} \vdash \square \Rightarrow S \models F$.

Paso SLD: $\leftarrow A_1, \dots, A_k, \dots, A_n$ con regla $A \leftarrow B_1..B_m$ y $\theta = \text{UMG}(A, A_k)$ da $\leftarrow (A_1..A_{k-1}, B_1..B_m, A_{k+1}..A_n)\theta$. Éxito: meta vacía $\square$.

Sintaxis: $,$ conjunción; $;$ disyunción (u otra respuesta en consola); $\neg$ negación como falla; $!$ corte; is evalúa aritmética; $=$ solo unifica; $==$ compara valores; Mayúscula inicial o $_$ = variable; minúscula = átomo; sin espacio entre predicado y paréntesis.

Tema 1. Fundamentos y técnicas de programación

Problema 1.1

¿Qué significa que un programa sea *correcto*? Explica por qué “correcto” y “eficiente” no son sinónimos y menciona tres criterios con los que suele juzgarse un programa.

Problema 1.2

Define *algoritmo* y *programa*. ¿Qué expresa la fórmula de Niklaus Wirth?

Problema 1.3

Sea $G = (V, T, S, P)$ con $T = \{a, b\}$, $N = \{S\}$ y producciones $S \rightarrow aSb \mid ab$.

- (a) Explica qué es cada componente de G .
- (b) Deriva la cadena $aaabbb$. ¿Cuál es $L(G)$?
- (c) ¿De qué tipo es G en la jerarquía de Chomsky? ¿Y la gramática $S \rightarrow aS \mid b$?

Problema 1.4

Clasifica por paradigma (imperativo / declarativo funcional / declarativo lógico) y por forma de ejecución típica: C, Pascal, Java, Haskell, Lisp, Prolog, Python. ¿Por qué la clasificación compilado/interpretado no es absoluta?

Problema 1.5

Enuncia el teorema de Böhm–Jacopini y da un ejemplo en pseudocódigo de cada estructura.

Problema 1.6 (★ pregunta del examen del año pasado)

¿Qué métricas de módulos existen para medir su eficiencia (y calidad)?

Problema 1.7

Las diapositivas advierten que “alta cohesión $\Rightarrow$ alto acoplamiento” no es la meta. Explica la idea y describe cómo MVC reparte responsabilidades entre módulos.

Tema 2. Divide y vencerás

Problema 2.1 (★ pregunta del examen del año pasado)

Escribe en pseudocódigo el algoritmo de *Divide y vencerás* para resolver un problema y explica las condiciones que debe cumplir el problema para aplicarlo.

Problema 2.2

Escribe la búsqueda binaria recursiva sobre un arreglo ordenado $A[izq..der]$. Traza la búsqueda de $X = 23$ y de $X = 10$ en $A = [3, 8, 12, 17, 23, 31, 40, 52]$ (índices 1..8). ¿Por qué las llamadas usan $m - 1$ y $m + 1$? ¿Cuál es su complejidad?

Problema 2.3

Diseña con divide y vencerás una función $\text{Maximo}(A, l, r)$ que devuelva el mayor elemento de $A[l..r]$. Plantea la recurrencia y resuélvela.

Problema 2.4

Considera el procedimiento $\text{Sort}(l, r)$ de Qsort.PAS :

```
i := 1; j := r; x := a[(1+r) div 2];
repeat
  while a[i] < x do i := i + 1;
  while x < a[j] do j := j - 1;
  if i <= j then begin
    y := a[i]; a[i] := a[j]; a[j] := y; i := i + 1; j := j - 1;
  end;
until i > j;
if l < j then Sort(l, j);
if i < r then Sort(i, r);
```

Traza la primera partición para $a = [5, 3, 8, 1, 9, 2, 7]$ (índices 1..7) y di qué llamadas recursivas se generan. ¿Qué enseña este ejemplo sobre el pivote? Da la complejidad en el mejor y peor caso.

Problema 2.5

Multiplica $u = 1234$ y $v = 5678$ con el esquema de divide y vencerás de clase ($s = 2, B = 10^2$). Da la recurrencia del esquema y explica la mejora de Karatsuba.

Problema 2.6

Escribe *mergesort* siguiendo el esquema del profesor, señalando qué parte es “descomponer” y cuál “combinar”. Compáralo con QuickSort: ¿dónde hace cada uno el trabajo pesado?

Problema 2.7

Responde brevemente: (a) ¿Todo algoritmo recursivo es divide y vencerás? (b) ¿Qué diferencia hay entre caso base y caso vacío? (c) ¿Cuál es la diferencia esencial entre divide y vencerás y backtracking?

Tema 3. Backtracking, ocho reinas y cuadros mágicos

Problema 3.1 (★ pregunta del examen del año pasado)

Describe el algoritmo de *Backtracking* cuando se trata de buscar la **mejor** solución de un problema. Indica en qué difiere de las versiones que buscan *una* y *todas* las soluciones.

Problema 3.2

Problema de las 4 reinas (una reina por fila, filas $k = 0..3$, columnas $j = 1..4$).

- ¿Por qué colocar una reina por fila elimina la restricción de filas sin comprobarla?
- ¿Qué identifican $j - k$ y $j + k$? ¿Se atacan $(k, j) = (0, 2)$ y $(2, 4)$? ¿Y $(1, 4)$ y $(3, 2)$?
- Traza el árbol de búsqueda en profundidad (probando columnas de 1 a 4) hasta la primera solución.
- ¿Cuántas soluciones tiene el problema de 4 reinas? ¿Y el de 8? ¿Qué son las 12 soluciones fundamentales?

Problema 3.3

En el programa `Ocho_reinas.c` visto en clase:

```
for (int j = 1; j <= NREINAS; j++)
    if (!contiene(col, j) && !contiene(diag45, j - k) && !contiene(diag135, j + k)) {
        sol[k] = j;
        col.push_back(j); diag45.push_back(j - k); diag135.push_back(j + k);
        reinas(k + 1, col, diag45, diag135);
        col.pop_back(); diag45.pop_back(); diag135.pop_back();
    }
```

- Relaciona cada línea con las operaciones del esquema de backtracking del profesor.
- ¿Qué representa $k == NREINAS$?
- ¿Por qué no hace falta “borrar” `sol[k]`?

Problema 3.4 (estilo examen)

Mochila 0/1: objetos con pesos $w = (2, 3, 4, 5)$ y valores $v = (3, 4, 5, 7)$, capacidad 7. Se quiere el subconjunto de mayor valor.

- Formúlalo como backtracking de *mejor solución*: ¿qué es una etapa, una opción, Aceptable, SolucionCompleta y EsMejorSolucion?
- Encuentra la mejor solución y muestra al menos una rama podada.

Problema 3.5

Cuadros mágicos.

- Deduce la constante mágica M_n y calcula M_3, M_4, M_5, M_6 .
- Construye el cuadro 3×3 y el 5×5 con el método siamés.
- Construye el 4×4 con el método de complementos.

(d) ¿Por qué el método siamés no sirve para $n = 4$ ni el de complementos para $n = 6$?

Problema 3.6

Formula la búsqueda de *todos* los cuadros mágicos 3×3 como backtracking: estado, opciones, condición de aceptación (poda) y solución completa. ¿Por qué para $n \geq 5$ es preferible la construcción por patrones si solo se quiere uno?

Tema 4. Lógica de Hoare: aserciones, asignación y consecuencia

Problema 4.1

Define: aserción, estado, $s \models P$, terna de Hoare, corrección parcial y corrección total. ¿Por qué en un fragmento sin ciclos ni recursión basta la corrección parcial?

Problema 4.2

Ordena de la más fuerte a la más débil: $x \geq 0$, $\top$, $x > 10$, $\perp$, $x = 15$, $x > 0$. ¿Qué significa la aserción vacía $\{\}$? ¿Es cierto que $y \neq 0 \Rightarrow y = 4$?

Problema 4.3

Calcula la precondition más débil P en cada caso:

- (a) $\{P\} x := x + 5 \{x > 12\}$ (b) $\{P\} y := 3x - 1 \{y \leq 8\}$
 (c) $\{P\} x := x \cdot x \{x < 9\}$ (d) $\{P\} z := z/2 \{z = 5\}$
 (e) $\{P\} a := b \{a = b\}$ (f) $\{P\} x := 10/(y - 2) \{x > 0\}$

Problema 4.4

¿Son válidas? Justifica con el axioma de asignación y la regla de consecuencia, o da un contraejemplo.

- (a) $\{x = 5\} x := x + 1 \{x > 5\}$ (b) $\{x > 0\} x := x - 1 \{x > 0\}$
 (c) $\{\top\} y := x \cdot x \{y \geq 0\}$ (d) $\{x \geq 0\} x := x - 1 \{x \geq -1\}$

Problema 4.5

(a) A partir de $\{y \neq 0\} x := 1/y \{x = 1/y\}$ deduce $\{y = 4\} x := 1/y \{x = 1/y\}$ escribiendo la regla usada.
 (b) A partir de $\{\top\} max := b \{max = b\}$ deduce $\{\top\} max := b \{max \geq b\}$. (c) Explica con conjuntos de estados por qué se puede *fortalecer* la precondition y *debilitar* la postcondición, pero no al revés.

Problema 4.6

Usa la regla de conjunción (y consecuencia) para demostrar $\{i = i_0 \wedge i_0 \geq 3\} i := i + 2 \{i \geq 5\}$, partiendo de las ternas $\{i = i_0\} i := i + 2 \{i = i_0 + 2\}$ y $\{i_0 \geq 3\} i := i + 2 \{i_0 \geq 3\}$. ¿Por qué la segunda terna es válida?

Tema 5. Composición secuencial e invariantes de bloque

Problema 5.1

Demuestra $\{ \} c := a + b; c := c/2 \{c = (a + b)/2\}$ (ejercicio del “tour” del curso).

Problema 5.2

Encuentra la precondition más débil: (a) $\{P\} x := x + 1; y := 2x \{y > 10\}$; (b) $\{P\} y := x - 3; z := y \cdot y \{z > 16\}$.

Problema 5.3

Demuestra que el intercambio *sin variable auxiliar* es correcto: $\{a = A \wedge b = B\} a := a + b; b := a - b; a := a - b \{a = B \wedge b = A\}$.

Problema 5.4

Demuestra que $I : s = i^2$ es invariante del bloque $i := i + 1; s := s + 2i - 1$. Comprueba con $i = 3, s = 9$.

Problema 5.5

Demuestra $\{ \} s := 1; s := s \cdot r + 2; s := s \cdot r + 3 \{s = r^2 + 2r + 3\}$ (evaluación de Horner). Observa dónde es necesaria la regla de consecuencia.

Tema 6. Condicionales

Problema 6.1 (★ pregunta del examen del año pasado)

Demostrar que la siguiente terna de Hoare es válida:

$$\{a < b\} \text{ if } (a > b) \text{ then } m := a \text{ else } m := b \{m = b\}.$$

Problema 6.2

Demuestra $\{ \} \text{ if } a > b \text{ then } m := a \text{ else } m := b \{m \geq a \wedge m \geq b\}$.

Problema 6.3 (estilo examen)

Demuestra $\{a > b\} \text{ if } (a < b) \text{ then } m := b \text{ else } m := a \{m = a\}$.

Problema 6.4

¿Es válida $\{ \} \text{ if } a > b \text{ then } m := a \text{ else } m := b \{m > b\}$? Si no, da un contraejemplo y propón dos formas de corregirla (cambiando solo la pre o solo la postcondición).

Problema 6.5

Demuestra $\{ \} \text{ if } x < 0 \text{ then } y := -x \text{ else } y := x \{y \geq 0 \wedge y^2 = x^2\}$ (valor absoluto).

Problema 6.6

(if sin else) Demuestra $\{ \} \text{ if } x > y \text{ then } (t := x; x := y; y := t) \{x \leq y\}$.

Problema 6.7

Demuestra $\{x > 0\} \text{ if } x > 10 \text{ then } x := x - 10 \text{ else } x := x + 5 \{x > 0\}$.

Problema 6.8 (reto)

Demuestra que el máximo de tres valores es correcto:

if $a \geq b$ **then** (**if** $a \geq c$ **then** $m := a$ **else** $m := c$) **else** (**if** $b \geq c$ **then** $m := b$ **else** $m := c$)

con $P = \top$ y $Q : m \geq a \wedge m \geq b \wedge m \geq c$.

Tema 7. Ciclos: inducción, invariantes y terminación**Problema 7.1** (★ pregunta del examen del año pasado)

Demostrar que el invariante en las siguientes instrucciones es $i + j = 9$:

```
int j = 9;
for( int i=0; i<10; i++ )
    j--;
```

¿Qué valores tienen i y j al terminar? ¿Por qué termina?

Problema 7.2 (estilo examen)

Encuentra y demuestra el invariante que relaciona k e i , y el valor final de k :

```
int k = 20;
for( int i = 0; i < 5; i++ )
    k = k - 4;
```

Problema 7.3

Demuestra que la función calcula el cuadrado de un entero $A \geq 0$ (identifica invariante, salida y variante):

```
Function Cuadrado( A )
    C ← 0; D ← 0
    While ( D ≠ A )
        C ← C + A
        D ← D + 1
    Return C
```

¿Qué ocurre si $A < 0$?

Problema 7.4

¿Qué realiza la subrutina? Demuéstralo (N, M enteros, $M \geq 0$).

```
Subroutine ?( N, M; R )
  R ← 1
  While ( M > 0 )
    R ← R × N
    M ← M - 1
  Return
End Subroutine
```

¿Por qué $R = N^M$ no es un invariante?

Problema 7.5

Demuestra la corrección total de ($n \geq 0$):

```
s := 0; i := 0;
while i ≠ n do
  i := i + 1;
  s := s + i
```

con postcondición $s = \frac{n(n+1)}{2}$.

Problema 7.6

División entera por restas sucesivas ($a \geq 0, b > 0$):

```
q := 0; r := a;
while r ≥ b do
  r := r - b;
  q := q + 1
```

Demuestra que al terminar $q = a \text{ div } b$ y $r = a \text{ mód } b$. Traza con $a = 17, b = 5$.

Problema 7.7

Para cada ciclo, propón el invariante y demuéstralo por inducción ($n \geq 0$): (a) $f:=1; i:=0; \text{ while } i < n \text{ do } \{ i:=i+1; f:=f*i \}$ (b) $s:=0; i:=0; \text{ while } i < n \text{ do } \{ s:=s+2*i+1; i:=i+1 \}$

Problema 7.8

Responde: (a) ¿Por qué una tabla de valores no es una demostración? (b) ¿Qué relación hay entre el caso base/paso inductivo y las obligaciones de Hoare para **while**? (c) ¿Qué se obtiene si se prueba el invariante y la salida, pero no la variante?

Tema 8. Función 91 de McCarthy, inducción modificada y Ackermann

Problema 8.1

Con $f(x) = x - 10$ si $x > 100$ y $f(x) = f(f(x + 11))$ si $x \leq 100$: (a) desarrolla a mano $f(99)$ y $f(95)$ y cuenta cuántas llamadas se hacen en cada caso; (b) desarrolla $f(89)$ mostrando solo los pasos intermedios importantes; (c) calcula $f(105)$ y $f(101)$.

Problema 8.2

Enuncia el principio de inducción matemática modificado y úsalo para demostrar que $f(x) = 91$ para todo entero $x \leq 100$. Explica por qué la inducción simple ("si vale en n vale en $n + 1$ ") no basta.

Problema 8.3 (estilo examen)

Sea $g(x) = x - 2$ si $x > 20$ y $g(x) = g(g(x + 3))$ si $x \leq 20$. Calcula $g(20)$, $g(19)$, $g(15)$; conjetura el valor de $g(x)$ para $x \leq 20$ y demuéstalo con inducción modificada.

Problema 8.4

Con la definición $A(0, y) = y + 1$, $A(x + 1, 0) = A(x, 1)$, $A(x + 1, y + 1) = A(x, A(x + 1, y))$: (a) calcula $A(1, 2)$ y $A(2, 1)$ aplicando *solo* la definición; (b) demuestra por inducción sobre z que $A(1, z) = z + 2$ y $A(2, z) = 2z + 3$.

Problema 8.5

Demuestra que $A(3, z) = 2^{z+3} - 3$. Calcula $A(3, 3)$ y $A(4, 1)$. ¿Por qué el programa recursivo de Ackermann termina en `StackOverflowException` para valores pequeños como $(4, 2)$?

Problema 8.6

Usando los lemas del problema anterior, demuestra $\{x = 2\} w := A(x, y) + 1 \{w = 2y + 4\}$ y $\{x = 1\} w := A(x, y) \{w = y + 2\}$. ¿Por qué no basta el axioma de asignación?

Problema 8.7

Compara la inducción usada para ciclos con la usada para f (McCarthy) y Ackermann: dirección, hipótesis y qué se demuestra.

Tema 9. Programación lógica y Prolog

Problema 9.1

(a) Escribe como cláusula (disyunción de literales) y como implicación la regla `abuelo(X, Z) :- padre(X, Y), padre(Y, Z)`. (b) ¿Qué es una cláusula de Horn definida? ¿Y una cláusula objetivo? Representa la consulta `?- abuelo(juan, W)`.

Problema 9.2

Da el unificador más general o explica por qué no existe: (a) $p(X, b)$ y $p(a, Y)$; (b) $f(X, g(Y))$ y $f(h(Z), g(a))$; (c) $p(X, X)$ y $p(a, b)$; (d) $gusta(maria, X)$ y $gusta(Y, cine)$; (e) $q(f(X), X)$ y $q(Y, a)$; (f) $p(X)$ y $p(f(X))$.

Problema 9.3

Demuestra por **refutación** con resolución: (a) $S = \{\neg hombre(X) \vee mortal(X), hombre(socrates)\} \models mortal(socrates)$. (b) $S = \{P(X) \vee Q(X), \neg Q(X) \vee R(X), \neg R(X) \vee P(X)\} \models P(X)$.

Problema 9.4

Dado el programa

```
padre(juan, pedro).
padre(pedro, luis).
padre(pedro, ana).
abuelo(X, Z) :- padre(X, Y), padre(Y, Z).
```

(a) Escribe la derivación SLD para $?- abuelo(juan, W)$. indicando en cada paso la cláusula usada y el unificador. (b) ¿Qué responde Prolog al teclear ;?

Problema 9.5

Explica por qué el paso “reemplazar una meta por el cuerpo de una regla” es una instancia de la regla de resolución. ¿Qué significan las letras S, L, D?

Problema 9.6

Dado el programa, indica si cada consulta tiene éxito, falla o produce error, y justifica:

```
a.
b.
c :- fail.
s :- a, \+ c.
t :- c ; b.
u :- a, c.
v :- \+ a ; c.
w :- b, ( \+ c, a ).
x :- y.
```

Consultas: $?- s.$ $?- t.$ $?- u.$ $?- v.$ $?- w.$ $?- x.$

Problema 9.7

Con el programa `MariaGusta.pl`

```
gusta(maria, helado).
gusta(maria, leer).
gusta(maria, cine).
dar(juan, beso, maria).
recibe(X, beso) :- dar(juan, beso, X).
```

indica la respuesta de: (a) gusta(maria, cine). (b) gusta(juan, cine). (c) gusta(Quien, helado). (d) gusta(maria, Maria). (con ;) (e) recibe(X, beso). (f) recibe (X, beso).

Problema 9.8 (estilo examen)

Considera

```
maximo( X, Y, X ) :- X >= Y.
maximo( X, Y, Y ) :- X < Y.
mayor( X, Y, X ) :- X >= Y, !.
mayor( X, Y, Y ) :- X < Y.
max( X, Y, Y ) :- X < Y, !.
max( X, _, X ).
```

(a) ¿Qué hace el corte !? (b) ¿Qué es un corte “verde” y uno “rojo”? Clasifica los de mayor y max. (c) ¿Qué responde ?- max(3, 5, 3) .? ¿Es correcto? Corrígelo.

Problema 9.9

Indica la respuesta de SWI-Prolog: (a) $X = 2+3$. (b) $X \text{ is } 2+3$. (c) $5 \text{ is } 2+3$. (d) $2+3 = 5$. (e) $2+3 \text{ := } 5$. (f) $X \text{ is } Y+1$.

Problema 9.10

(a) Traza ?- factorial(3, F). con

```
factorial(0, 1) :- !.
factorial(N, F) :- N > 0, N1 is N - 1, factorial(N1, F1), F is N * F1.
```

(b) Escribe suma_hasta(N, S) ($S = 1 + 2 + \dots + N$) y mcd(A, B, G) (algoritmo de Euclides) con el mismo estilo.

Problema 9.11

Compara resolución general y resolución SLD (qué cláusulas usan, forma de la derivación, qué significa el éxito, cómo manejan alternativas). ¿Por qué el orden de las cláusulas importa en Prolog aunque lógicamente no importe?

Tema 10. Exámenes simulados

Resuelve cada simulacro en **90 minutos**, sin notas, escribiendo las demostraciones completas como si fuera el examen real. Los problemas se numeran como **10.1–10.6** (simulacro A) y **10.7–10.12** (simulacro B).

Simulacro A

Problema 10.1 · A1

Demostrar que la siguiente terna de Hoare es válida:

$$\{a \neq b\} \text{ if } (a > b) \text{ then } (m := a; n := b) \text{ else } (m := b; n := a) \{m > n\}.$$

Problema 10.2 · A2

¿Qué es la corrección parcial y la corrección total de un programa? Da un ejemplo de programa parcialmente correcto que no sea totalmente correcto.

Problema 10.3 · A3

Escribe en pseudocódigo un algoritmo de divide y vencerás que calcule la suma de los elementos de $A[l..r]$. Plantea su recurrencia y traza la llamada sobre $A = [4, 7, 1, 3]$.

Problema 10.4 · A4

Demostrar que el invariante de las siguientes instrucciones es $s = 2i$ y deducir el valor final de s ($n \geq 0$):

```
int s = 0;
for( int i = 0; i < n; i++ )
    s = s + 2;
```

Problema 10.5 · A5

Con el programa

```
progenitor(ana, bea).
progenitor(ana, carlos).
progenitor(carlos, diana).
progenitor(bea, emilio).
abuela(X, Z) :- progenitor(X, Y), progenitor(Y, Z).
```

- Escribe la derivación SLD de $?- \text{abuela}(\text{ana}, N)$. y todas las respuestas que da Prolog con $;$.
- ¿Qué responde $?- \text{abuela}(X, \text{diana})$?

Problema 10.6 · A6

En el problema de las ocho reinas, explica para qué sirven $j-k$ y $j+k$, por qué basta el vector $\text{sol}[k]$ y qué parte del algoritmo realiza el “backtracking”.

Simulacro B

Problema 10.7 · B1

Encuentra la precondition más débil P tal que $\{P\} x := 2x + 1; y := x - 3 \{y > 4\}$ sea válida. ¿Es válida $\{x = 5\} x := 2x + 1; y := x - 3 \{y > 4\}$?

Problema 10.8 · B2

Describe el algoritmo de backtracking para encontrar *todas* las soluciones de un problema. Aplícalo al problema de las 4 reinas: ¿cuáles son las soluciones?

Problema 10.9 · B3

Suponiendo demostrado que $A(1, z) = z + 2$, demuestra que $A(2, z) = 2z + 3$ para todo $z \geq 0$, calcula $A(2, 5)$ y demuestra $\{x = 1\} w := A(x, y) \{w = y + 2\}$.

Problema 10.10 · B4

Para el ciclo de división entera por restas ($a \geq 0, b > 0$)

```
q := 0; r := a;
while r ≥ b do { r := r - b; q := q + 1 }
```

encuentra el invariante, demuéstalo por inducción y prueba la corrección total respecto de $Q : a = qb + r \wedge 0 \leq r < b$.

Problema 10.11 · B5

(a) ¿Qué responde Prolog a $?- \text{max}(3, 5, 3)$. con $\text{max}(X, Y, Y) :- X < Y, !.$ y $\text{max}(X, _, X) .$? Explica y corrige. (b) ¿Qué diferencia hay entre $X = 3*2$, $X \text{ is } 3*2$ y $3*2 == 6$?

Problema 10.12 · B6

Demuestra con el principio de inducción modificado que la función 91 de McCarthy cumple $f(x) = 91$ para todo entero $x \leq 100$.

Lista de verificación antes del examen

- Sé escribir de memoria: esquema de divide y vencerás; backtracking para una, todas y la mejor solución.
- Sé enunciar: axioma de asignación, consecuencia, composición, **if** con y sin **else**, obligaciones del **while**, inducción ordinaria y modificada.
- Reconozco una rama inalcanzable ($P \wedge B \equiv \perp$) y sé justificarla.
- Sé encontrar un invariante trazando 3–4 iteraciones y eliminando n ; siempre incluyo salida y variante.
- Sé derivar $A(1, z)$, $A(2, z)$, $A(3, z)$ y la prueba de $f(x) = 91$.
- Sé pasar reglas de Prolog a cláusulas, unificar, hacer una derivación SLD y predecir la consola

(;, !, \+, is, mayúsculas).

- Sé definir cohesión, acoplamiento y métricas de eficiencia (tiempo, memoria).